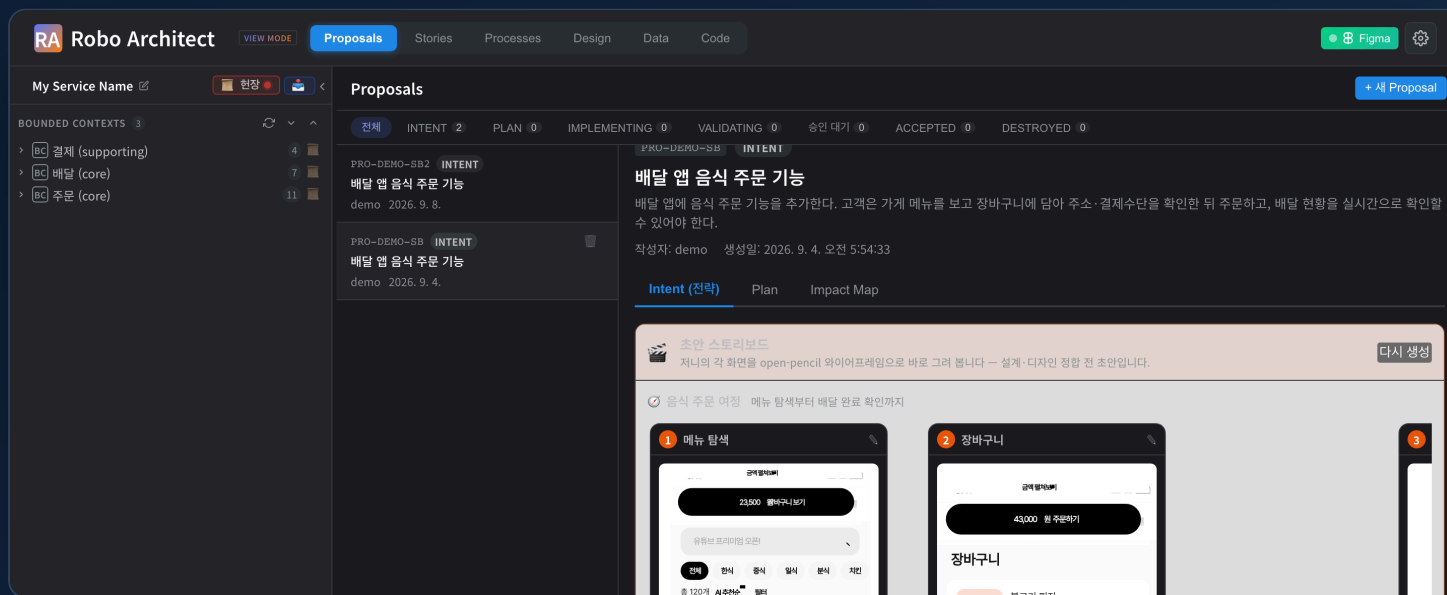


제 품 소 개 및 도 입 제 안 서

요구사항이 바뀌면, 설계도 함께 움직입니다.

요구사항을 도메인 그래프로 구조화하고, 변경에 따른 영향 범위를 분석하며, 화면과 코드까지 하나의 흐름으로 연결하는 **AI 아키텍처 워크벤치**입니다.



요구사항 → 코드

한 흐름으로 연결

22종

도메인 그래프 노드

3모드

간편 · DDD 상세 · ODA 표준

2026년 9월

AI 아키텍처 워크벤치 · 온프레미스 / 데스크톱 설치

uEngine

목차

01 왜 지금인가 — 설계의 문제	3	09 특징점 ⑨ 초안에서 이미 화면이 보인다	12
02 Robo Architect 소개	4	10 특징점 ⑩ 승인 후 반영되는 변경	13
03 주요 산출물 ① 도메인 그래프	5	11 아키텍처 — 시스템 구성과 실행 환경	14
04 주요 산출물 ② 화면과 코드	6	12 기술 스택	15
05 제품 화면 한눈에 보기	7	13 사용 흐름 (실제 화면)	16-17
06 특징점 ①② 그래프 · 영향 분석	8-9	14 보안 · 거버넌스 · 납품	18-19
07 특징점 ③ 제안부터 병합까지	10	15 검증 체계	20
08 특징점 ④ 방법론이 제품 안에	11	16 도입 실적 · 도입 절차	21-22

한 장 요약

요구사항과 코드가 바뀔 때마다 설계 문서도 함께 갱신해야 합니다. Robo Architect는 설계를 **문서가 아니라 그래프로** 관리해, 요구사항이 바뀌면 영향 범위를 분석해 보여주고 승인된 변경 사항을 코드와 그래프에 함께 반영합니다.

그래프가 기준

요구사항, 설계 요소, 화면, 소스 파일의 연결 관계를 하나의 그래프로 관리합니다

한 흐름

제안 → 분해 → 계획 → 구현 → 검증 → 병합이 끊기지 않습니다

폐쇄망

데스크톱 설치본으로 납품되며 사내 LLM 게이트웨이를 씁니다

핵심 차이

설계 도구나 코드 생성기와 달리, **설계와 코드 구현을 연결하고 변경 이후에도 둘의 관계를 추적**할 수 있도록 지원합니다. AI가 코드를 작성하기 전에 무엇을 만들지 도메인 언어로 정리하고, 코드가 만들어진 뒤에는 생성된 소스와 설계 요소의 연결 관계를 그래프에 기록합니다.

이 문서가 다루는 것

제품 개요

3~13쪽
문제 · 정의 · 산출물 · 특징점

시스템 구성

14~15쪽
토폴로지 · 기술 스택

사용 절차

16~17쪽
실행 중인 제품 화면 · 6단계

도입 안내

18~22쪽
보안 · 납품 · 레퍼런스 · 일정

왜 지금인가 — 설계의 문제

AI가 코드 작성을 지원하면서, **무엇을 만들고 어떻게 설계할지 결정하는 일**이 더욱 중요해졌습니다. 하지만 그 결정은 여전히 문서에 머물러 있어, 이후 변경 사항을 지속적으로 반영하기 어렵습니다.

문제 1

설계 문서가 코드의 변경을 따라가지 못한다

이벤트스토밍 결과를 문서로 정리하고 나면, 이후 변경은 문서에 반영되지 않는 경우가 많습니다. 시간이 지나면 문서는 과거의 설계만 담게 되어, 현재 시스템의 구조를 파악하기 어려워집니다.

문제 2

"이 기능을 바꾸면 어디에 영향이 있나요"

가장 자주 나오고 가장 답하기 어려운 질문입니다. 담당자의 기억에 의존하거나 관련 코드를 일일이 확인해야 하며, 영향 범위를 놓치면 배포 후 오류로 이어질 수 있습니다.

문제 3

AI에게 전달할 맥락을 매번 다시 정리해야 한다

코딩 에이전트는 지시한 작업을 정확히 수행합니다. 다만 이 변경이 우리 도메인에서 어떤 의미인지는 대화할 때마다 사람이 다시 설명해야 합니다.

문제 4

방법론이 실무에 정착하지 않는다

DDD · 이벤트스토밍 교육을 받아도, 프로젝트에서는 화이트보드 사진 몇 장으로 끝나는 경우가 많습니다. 방법론을 지원하는 도구가 없으면 다음 프로젝트에서도 일관되게 적용하기 어렵습니다.

그래서 필요한 것

설계를 **지속적으로 갱신되는 그래프**로 관리하고, 요구사항이 바뀔 때 영향 범위를 분석해 보여주며, 승인된 변경 사항만 코드와 그래프에 **함께** 반영하는 도구.

기업에서 사용하려면 세 가지를 함께 고려해야 합니다. 소스와 설계 데이터가 외부로 나갈 수 없는 폐쇄망 환경, 기존에 쓰던 Figma · Jira · Confluence · Git과의 연동, 그리고 AI가 만든 산출물의 검증입니다. Robo Architect는 이 세 가지를 전제로 설계했습니다.

Robo Architect 소개

요구사항 분석부터 설계, 화면 구성, 코드 구현까지의 과정을 하나의 도메인 그래프로 연결하는 **AI 아키텍처 워크벤치**입니다. 요구사항 문서를 업로드하면 이벤트스토밍 기반의 설계 그래프를 생성합니다. 변경을 제안하면 영향 범위와 구현 계획을 제시하고, 승인된 변경 사항을 코드와 설계 그래프에 함께 반영합니다.

입력

요구사항

기획 문서(PDF · 텍스트),
Confluence 페이지, Figma
화면을 올리거나, 구현할 기능을
글로 직접 입력합니다.

처리

도메인 그래프로 구조화

요구사항을 Bounded Context ·
Aggregate · Command · Event
· Policy · 화면 등 DDD 설계
요소와 그 관계로 구조화해 그래프
데이터베이스에 저장합니다.

출력

설계 · 화면 · 코드

영향 분석, 구현 계획,
와이어프레임, 그리고 실제 소스
변경까지 이어집니다.

기존 방식과의 차이

	기존 설계 도구 · 문서	Robo Architect
설계의 형태	다이어그램 이미지와 문서	질의 가능한 그래프(노드 22종 · 관계 32종)
변경 영향	사람이 전수조사	그래프 탐색으로 영향 노드 산출
화면	별도 도구에서 따로 관리	같은 그래프의 UI 노드 — 초안 단계부터 생성
코드	설계와 단절	구현 파일과 설계 요소의 연결 관계를 기록
방법론	교육 자료	단계별 절차로 제품에 내장(DDD 6단계 · ODA 표준)
AI의 역할	코드 자동완성	구체화 · 계획 · 구현 · 검증을 사용자 승인 아래 수행

한눈에 보는 흐름



사용자가 단계별 결과를 확정하면 다음 단계로 진행하며, **승인된 변경 사항만** 반영됩니다.

주요 산출물 ① — 도메인 그래프

요구사항 문서를 업로드하면 **21단계 처리 과정**을 거쳐 요구사항에서 도메인 개념과 관계를 추출하고 그래프에 저장합니다. 진행 상황은 단계별로 표시되며, 중간에 멈춰 검토한 뒤 이어서 진행할 수 있습니다.



그래프에 쌓이는 것

계층	노드
요구사항	Requirement · UserStory · Feature · Given/When/Then(인수조건)
전략 설계	BoundedContext · Policy
전술 설계	Aggregate · Command · Event · ReadModel · Property · Invariant
화면 · 흐름	UI · Journey · JourneyStep · FigmaBinding · StoryboardPageMapping
구현	ImplementationFile — 설계 요소가 어느 소스 파일로 구현됐는지

22종

노드 라벨

요구사항부터 구현 파일까지

32종

관계 타입

EMITS · TRIGGERS · IMPLEMENTS · EFFECT ...

21단계

문서 수집·분석 과정

단계별 진행 표시 · 중단/재개

왜 그래프여야 하는가. "이 이벤트를 발행하는 명령은?", "이 유저스토리를 구현하는 파일은?", "이 Aggregate를 바꾸면 영향받는 화면은?" — 이런 질문에 답하려면 관계를 따라가야 합니다. 그래프를 사용하면 설계 요소 간 관계를 따라 변경의 영향 범위와 구현 위치를 확인할 수 있습니다.

설계 정보의 단일 기준

제품 개발 원칙의 첫 항목은 "그래프가 유일한 기준"입니다. 화면에 보이는 내용과 내보내는 문서는 모두 그래프에서 **다시 생성할 수 있는 산출물**이며, 별도의 기준 데이터를 만들지 않습니다.

주요 산출물 ② — 화면과 코드

설계만 산출되는 것이 아닙니다. 같은 그래프에서 **화면**을 생성하고, 그 설계를 근거로 **코드**를 작성하며, 생성된 소스 파일과 설계 요소의 연결 관계를 그래프에 기록합니다.

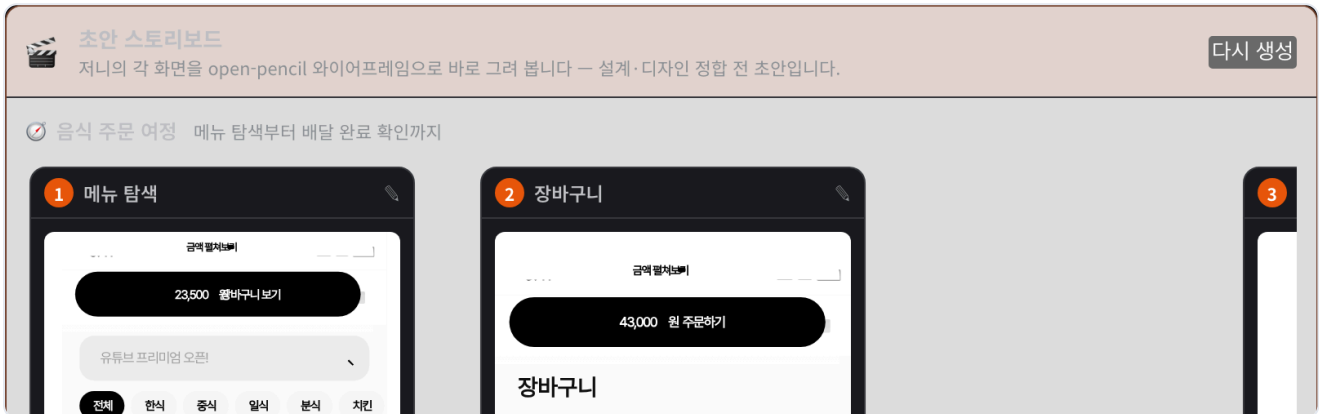


그림 2. 제안 초안 단계에서 자동 생성된 스토리보드 — 사용자 흐름의 각 화면을 와이어프레임으로 그려 흐름 순서대로 배치합니다. 마름모는 조건 분기입니다. *데모 데이터 기준입니다.*

화면 — 그림이 아니라 편집 가능한 구조

와이어프레임은 이미지가 아니라 요소 트리입니다. 그래서 그 자리에서 편집기를 열어 고치고, Figma로 내보내 디자이너가 이어받을 수 있습니다. 사내 디자인 시스템의 컴포넌트를 그대로 배치합니다.

코드 — 격리된 작업 공간에서

구현은 대상 프로젝트의 **격리된 작업 공간**(git worktree)에서 진행되며, 원본 브랜치는 승인 시점에만 병합합니다. 구현 도중 작업을 중단하고 수정 지시를 전달한 뒤 다시 진행할 수 있습니다.

문서로 내보내기

산출물	내용
PRD · 기능명세	Bounded Context 단위로 요구사항·설계·인수조건을 묶어 생성
이벤트 모델	UI → 명령 → 이벤트 → 읽기모델 스웸레인, GWT 시나리오 포함
BPMN 프로세스	업무 흐름을 표준 BPMN으로 내보내기
화면 · 스토리보드	와이어프레임, Figma 프레임, <code>.fig</code> 파일
사용자 매뉴얼	실제 화면을 캡처해 DOCX 매뉴얼로 자동 생성

모든 산출물은 그래프의 현재 상태를 기준으로 다시 생성됩니다. 문서를 직접 고치는 대신 그래프를 수정하고 문서를 다시 생성하므로, 설계와 문서 간 불일치를 줄일 수 있습니다.

제품 화면 한눈에 보기

같은 그래프를 역할에 맞는 네 개의 창으로 봅니다. 아래는 모두 실행 중인 제품에서 캡처한 화면입니다.

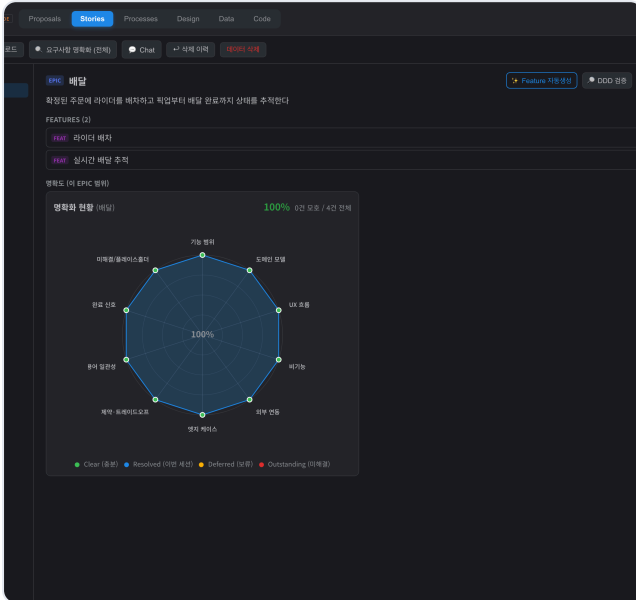


그림 3. Stories — Epic → Feature → UserStory → 인수조건 트리. 모호한 요구사항은 시가 되므로 답을 반영합니다.

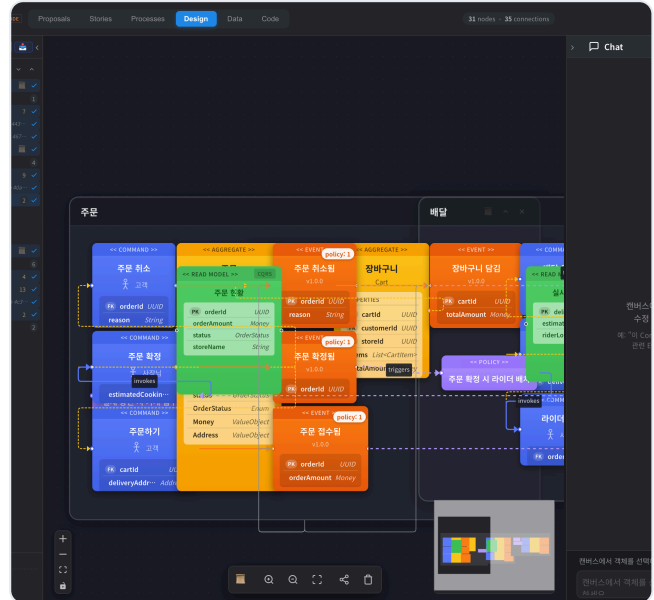


그림 4. Design — 이벤트스토밍 캔버스. 명령(파랑) · Aggregate(주황) · 이벤트 · 정책 · 읽기모델이 컨텍스트 경계 안에 배치됩니다.

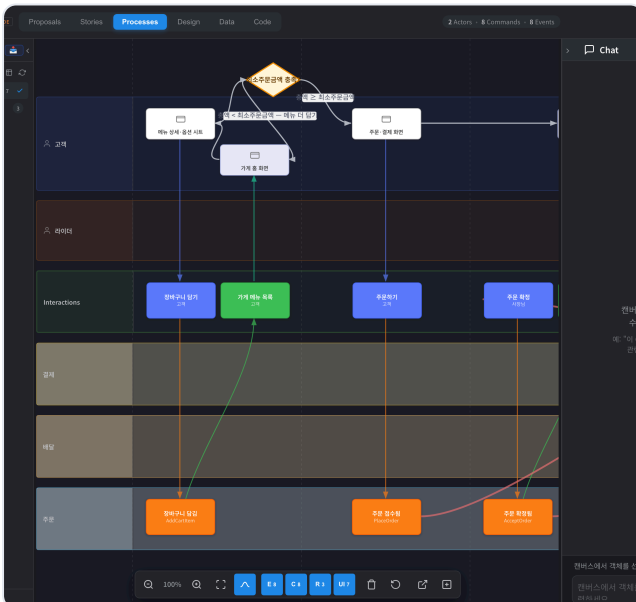


그림 5. Processes — 이벤트 모델링 swim레인. 액터 · 화면 · 명령 · 이벤트가 시간 순으로 흐르고, 조건 분기는 마름모로 표시됩니다.

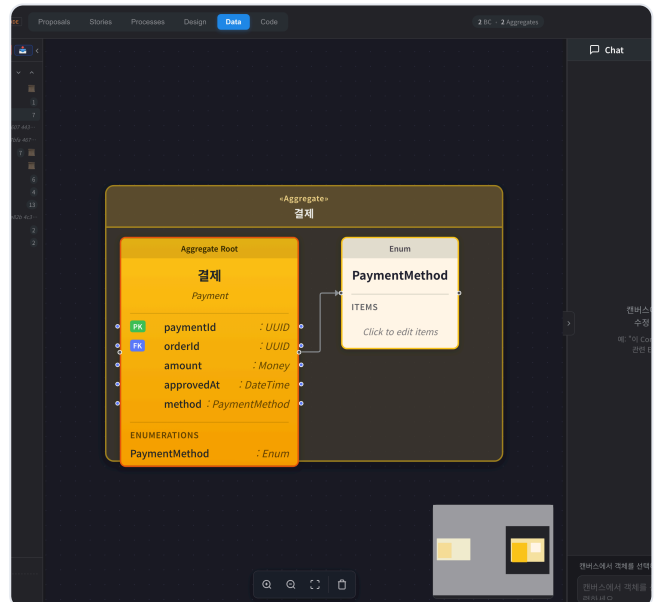


그림 6. Data — Aggregate의 속성 · 불변식 · 읽기 모델(CQRS)을 상세히 검토해 구현 단위를 확정합니다.

네 화면은 각각 다른 데이터를 보는 것이 아니라 **같은 그래프의 다른 단면**입니다. 한 곳에서 이름을 바꾸면 나머지 세 곳에 즉시 반영됩니다.

① 설계가 문서가 아니라 그래프입니다

대부분의 설계 도구는 그림을 그립니다. Robo Architect는 **관계를 저장**합니다. 화면에 보이는 다이어그램은 그 관계를 그려 낸 결과이며, 설계 정보의 기준은 언제나 그래프입니다.

그래서 가능해지는 것

- **질의** — "결제 실패 이벤트를 듣는 정책은 어디 있나", "이 읽기모델을 쓰는 화면은 몇 개인가"에 탐색으로 답합니다.
- **추적** — 요구사항 한 줄에서 그것을 구현한 소스 파일까지 경로가 이어집니다.
- **재생성** — 문서 · 다이어그램 · 화면이 모두 그래프에서 생성되므로, 그래프가 최신이면 산출물도 최신 상태로 유지됩니다.
- **협업** — 같은 그래프를 기획자는 요구사항 트리로, 아키텍트는 이벤트 모델로, 디자이너는 화면 흐름으로 봅니다.

역할별 화면 구성

기획 · PO

요구사항 트리

Epic → Feature → UserStory → 인수조건. 모호한 요구사항은 AI가 질문을 만들어 되묻고, 답을 요구사항에 반영합니다.

아키텍트

이벤트스토밍 캔버스

Bounded Context 경계, Aggregate, 명령 · 이벤트 · 정책을 한 화면에서 봅니다. 노드를 선택해 속성과 불변식을 확인하고 수정할 수 있습니다.

개발

Aggregate · CQRS 뷰

Aggregate의 속성 · 불변식 · 읽기 모델을 상세히 검토해 구현 단위를 확정합니다.

디자이너

화면 흐름 · Figma 연동

사용자 흐름과 화면을 확인하고, Figma 문서와 양방향으로 동기화합니다.

도메인 언어를 일관되게 사용합니다

제품 개발 원칙의 두 번째 항목은 "이벤트스토밍 어휘를 쓴다"입니다. 화면·API·데이터 모델 어디에서도

`create/update/delete` 같은 CRUD 어휘 대신 **명령과 이벤트의 도메인 이름**을 사용합니다. 용어가 일관되지 않으면 그래프의 연결도 어긋나기 때문입니다.

② 변경에 따른 영향 범위를 자동으로 분석합니다

요구사항 하나를 수정하면, 그 변경이 연결된 설계 요소에 어디까지 영향을 주는지 분석해 보여줍니다. 담당자의 기억에 의존하는 대신 **그래프의 관계를 따라** 영향 범위를 계산합니다.



영향 분석이 내놓는 것

항목	내용
영향 요소 목록	변경의 영향을 받는 설계 요소를 계층별로 제시하고, 확신도(높음 · 중간 · 낮음)와 판단 근거를 함께 표시합니다
충돌 가능성	동일한 설계 요소에 영향을 주는 다른 변경 요청이 있으면 함께 표시합니다
회귀 테스트 범위	영향 경로에 포함된 인수조건(GWT)을 모아 재검증 대상으로 제시합니다

미리보기는 원본 그래프를 수정하지 않고 변경 후의 구조를 보여줍니다. 기존 뷰어를 그대로 사용하므로 별도의 화면을 익힐 필요가 없고, 검토를 중단해도 정리할 작업이 남지 않습니다.

분석 결과는 이렇게 나옵니다

예시 — "주문 취소 시 환불 규칙을 바꾼다"는 요청

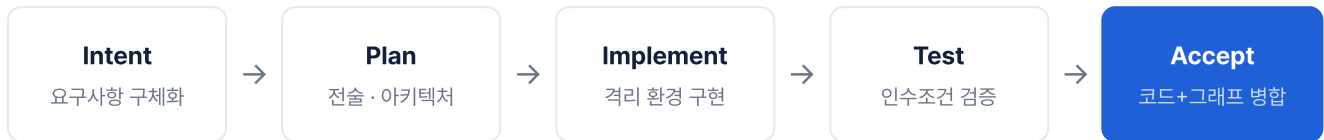
계층	영향받는 설계 요소
요구사항	주문 취소 · 환불 확인 관련 UserStory와 인수조건
전술 설계	주문 취소 명령, 주문 취소됨 이벤트, 환불 정책, 결제 불변식
화면	주문 취소 화면, 주문 현황 화면
구현	해당 설계 요소에 연결된 소스 파일

변경 이력

변경 요청마다 고유 번호를 부여하고, 영향을 받는 설계 요소와 연결해 기록합니다. 요청은 **초안 → 제출 → 승인 → 구현 완료** 순서로 진행됩니다. 요청자는 본인이 제출한 변경을 직접 승인할 수 없습니다. 변경 이력을 통해 설계가 언제, 어떤 이유로 수정되었는지 확인할 수 있습니다.

③ 변경 제안부터 구현 · 검증 · 병합까지

추가하거나 변경할 기능을 글로 입력하면 AI가 요구사항을 세분화하고 구현 계획을 제시합니다. 사용자는 단계별 결과를 검토하고, 승인한 계획에 따라 구현과 검증을 진행합니다. 최종 승인 후에는 구현 코드를 병합하고 설계 그래프에도 변경 사항을 반영합니다.



단계	하는 일
Intent	입력한 글을 Epic · Feature · UserStory · 사용자 흐름으로 구체화합니다. 내용이 모호하면 최대 5개의 선택형 질문으로 확인합니다
Plan	확정된 요구사항과 프로젝트 설계 원칙 (기술 스택 · 모놀리스/마이크로서비스 · 저장소 구성 전략)을 근거로 Aggregate · 명령 · 이벤트와 배포 · 연동 계획을 산출합니다
Implement	대상 프로젝트에 전용 브랜치의 격리된 작업 공간을 만들고, Code 탭에서 Claude Code가 구현을 수행합니다. 중지 · 수정 지시 · 재시도가 가능합니다
Test	인수조건(GWT)을 기준으로 결과를 검증하고 구조 검사를 함께 수행합니다
Accept	코드 병합과 설계 그래프 반영을 함께 수행합니다. 한쪽이 실패하면 양쪽을 되돌립니다

코드와 설계를 함께 반영합니다

승인하면 코드 병합과 설계 그래프 반영이 함께 진행됩니다. 어느 한쪽이 실패하면 양쪽을 되돌리고 실패 상태로 표시해 재시도할 수 있습니다. **코드만 병합되고 설계에는 변경 사항이 반영되지 않는 상황**을 방지하기 위한 구조입니다.

승인을 취소하면 설계 그래프의 반영을 되돌리며, 코드 되돌림 여부는 선택할 수 있습니다. 제안을 폐기하면 격리된 작업 공간도 함께 정리됩니다.

④ 방법론이 제품 안에 있습니다

DDD를 실무에 적용할 수 있도록 **단계별 절차와 검토 질문**을 제품에 내장했습니다. 제안을 만들 때 분해 모드를 선택하면 해당 모드의 절차가 그대로 진행됩니다.

세 가지 분해 모드

간편(Simplified)

요구사항 구체화와 계획 두 단계로 진행합니다. 영향 범위가 작고 신속한 처리가 필요한 변경에 적합합니다.

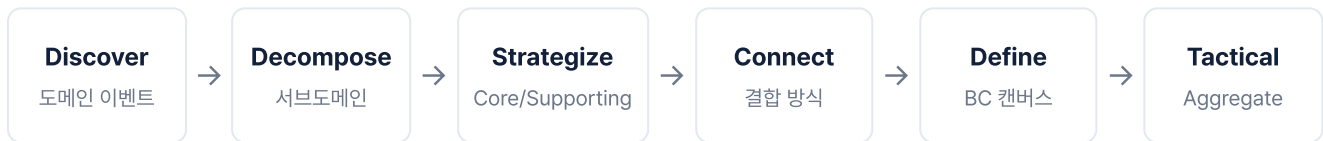
DDD 상세(Detailed)

ddd-starter 방법론의 6단계를 그대로 밟습니다. 새 도메인이나 큰 구조 변경에 적합합니다.

ODA 표준

TM Forum ODA / SID / Open API 표준에 근거해 분해하고 적합성을 점검합니다. 통신 도메인용입니다.

DDD 상세 모드의 6단계



단계마다 **설계 결정에 필요한 질문**을 제시합니다. "이 서브도메인이 사업의 핵심 경쟁력을 담당하는가?", "서비스 간 연동에 이벤트 방식을 사용할 것인가, 동기 호출을 사용할 것인가?", "이 Aggregate가 지켜야 할 불변식은 무엇인가?" 같은 질문에 선택형으로 답하고, 필요하면 이전 단계로 돌아가 다시 생성합니다. 먼저 **변경 범위를 분석**해 필요한 단계를 제안하므로, 작은 변경에 6단계를 모두 적용하지 않습니다.

전략적 결정은 한 번만 묻습니다. 비즈니스 차별성, 컨텍스트별 Core/Supporting/Generic 분류, 기본 결합 방식처럼 오래 유지되는 결정은 **프로젝트의 공통 설계 원칙**으로 저장되어, 다음 제안부터는 다시 묻지 않고 재사용합니다. 담당자가 바뀌어도 판단 기준이 조직에 남습니다.

절차는 스킬 파일로 관리합니다

각 단계는 독립된 **스킬**(AI의 작업 절차와 규칙을 정의한 파일, 총 18종)로 구현되어 있습니다. 방법론이 바뀌거나 조직 고유의 절차를 추가할 때는 스킬만 수정하면 되며, 제품 코드를 고칠 필요가 없습니다.

⑤ 초안에서 이미 화면이 보입니다

설계 검토에서 가장 어려운 순간은 "이게 실제로 어떤 화면이 되는지"를 상상해야 할 때입니다. Robo Architect는 요구사항을 기능과 사용자 흐름으로 구체화하는 단계가 끝나는 즉시 각 화면을 와이어프레임으로 그려 보여줍니다.

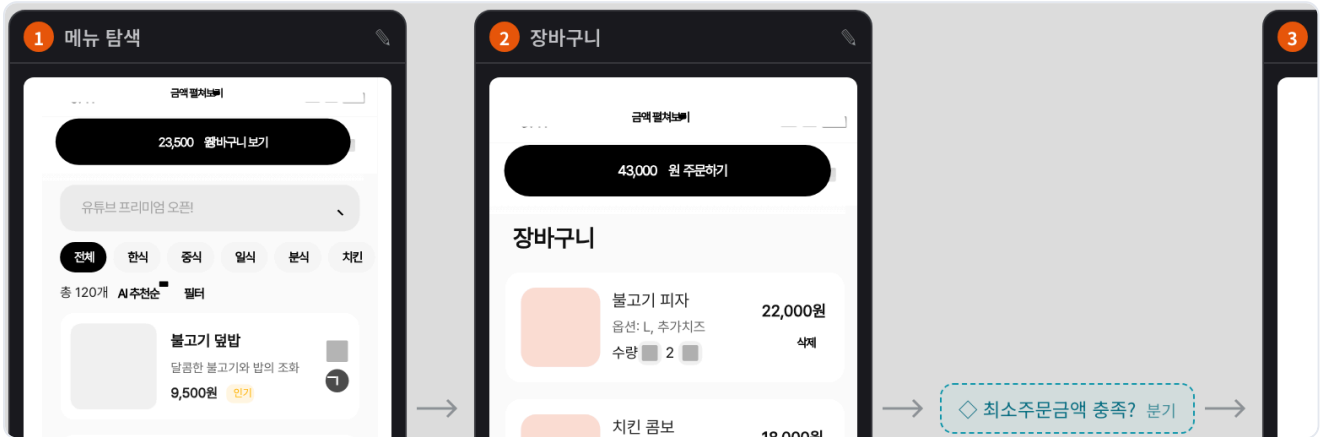


그림 7. 사용자 흐름의 화면이 순서대로 놓이고, 조건 분기는 마름모로 표시됩니다. 화면 카드는 그림이 아니라 편집 가능한 요소 트리입니다. 데모 데이터 기준입니다.

초안 단계에서 화면 확인

이전에는 설계와 디자인이 모두 끝난 뒤에야 화면을 확인할 수 있었습니다. 이제 제안 초안 단계에서 바로 확인합니다.

화면에서 바로 수정

카드의 편집 아이콘을 누르면 디자인 편집기가 열립니다. 요소를 옮기거나 크기를 바꾼 뒤 저장하면 해당 단계의 화면에 수정 사항이 반영됩니다.

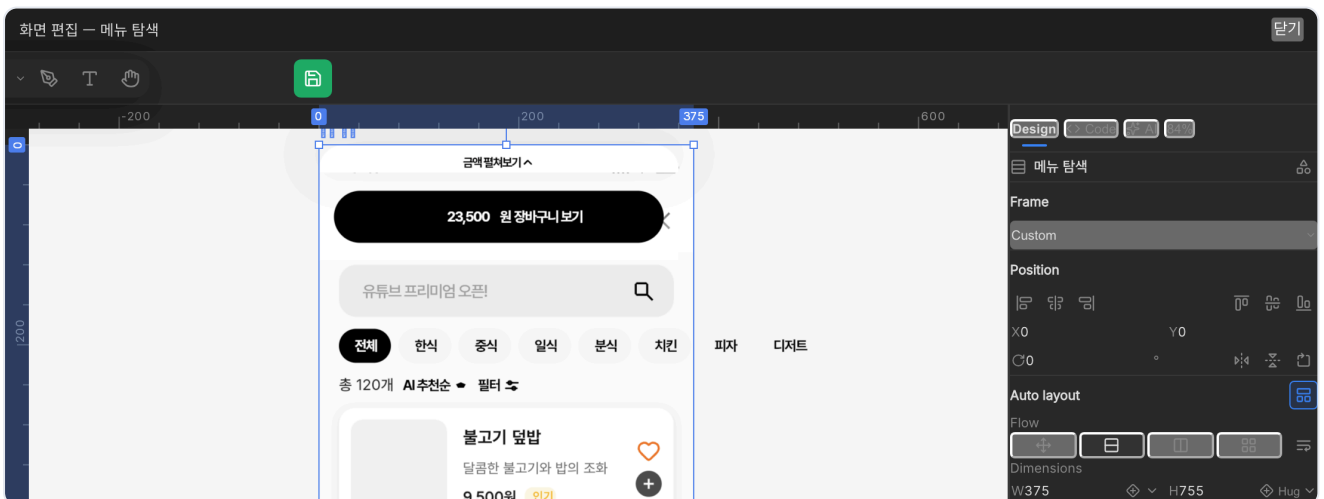


그림 8. 내장 디자인 편집기 — 캔버스 · 도구 모음 · 속성 패널, 좌표·정렬·오토레이아웃·색을 직접 조정할 수 있습니다.

사내 디자인 시스템을 적용할 수 있습니다. Figma 컴포넌트 라이브러리를 연결하면 기존 버튼 · 헤더 · 카드 등을 활용해 화면을 구성합니다. 컴포넌트의 유형과 문구를 화면에 맞게 설정하고, 내장 편집기에서 직접 수정할 수 있습니다.

⑥ 최종 변경 사항은 승인 후 반영됩니다

사용자는 AI가 제안한 변경이 **언제, 어디에 반영되는지** 미리 확인할 수 있어야 합니다. Robo Architect는 모든 변경을 **제안** → **확인** → **적용** 세 단계로 나눕니다.



- 분석은 아무것도 바꾸지 않습니다. 영향 분석·미리보기·스토리보드는 모두 읽기 전용입니다.
- 편집 제안과 적용이 분리돼 있습니다. 요구사항 명확화도 답변 → 편집안 제시 → 적용 승인 순서를 따릅니다.
- 동시 편집으로 인한 덮어쓰기를 방지합니다. 다른 사람이 먼저 수정했다면 충돌로 알리고 덮어쓰지 않습니다.
- 되돌릴 수 있습니다. 변경마다 이력이 남으며, 승인을 취소하면 설계 그래프의 반영을 되돌리고 코드 되돌림 여부는 선택합니다.

무엇이 언제 반영되는가

작업	대상 브랜치 · 설계 그래프에 최종 반영	설명
영향 분석 · 미리보기 · 스토리보드	없음	모두 읽기 전용입니다
제안 내부의 초안 편집	없음	제안에만 저장되며 운영 모델은 그대로입니다
격리 환경에서의 구현	없음	격리된 작업 공간의 파일만 바뀌며, 대상 브랜치는 그대로입니다
계획(Plan) 확정	없음	구현 계획만 확정합니다
최종 승인(Accept)	있음	코드 병합과 그래프 반영이 함께 일어납니다

진행 상황이 보입니다

수 초 이상 걸리는 모든 작업은 진행 상황을 실시간으로 전달합니다 — 문서 수집·분석 21단계, 제안 분해, 구현 로그, 화면 렌더링까지. AI가 **현재 수행 중인 작업과 그 이유**를 한국어로 기록하므로, 결과의 근거를 확인하기 어려운 상황이 생기지 않도록 설계했습니다.

모든 작업에 추적 번호가 붙습니다

요청마다 상관관계 ID가 부여되고 구조화 로그로 남습니다. 문제가 생기면 어느 단계에서 무엇이 실패했는지 로그로 되짚을 수 있습니다.

시스템 구성과 실행 환경

데스크톱 애플리케이션 하나에 **웹 UI · API 서버 · 그래프 데이터베이스 · 렌더 엔진**이 함께 들어 있습니다. LLM 요청의 전송 대상은 설정에서 지정하며, 사내 게이트웨이로 연결할 수 있습니다.



요청 처리 흐름

- 1 **브라우저 UI → API 서버** — 사용자가 문서를 올리거나 제안을 만듭니다.
- 2 **서버가 스킬을 실행** — 해당 단계의 스킬을 코딩 에이전트로 실행하고, 진행 상황을 화면에 실시간으로 전달합니다.
- 3 **결과는 그래프로** — 산출된 분해 · 설계는 Neo4j에 적재됩니다. 파일로 흩어지지 않습니다.
- 4 **화면이 필요하면 렌더 엔진** — JSX를 받아 레이아웃을 계산하고 화면 트리를 돌려줍니다.
- 5 **코드는 격리된 작업 공간에서** — 대상 프로젝트의 별도 작업 공간에서 구현하고, 승인 시 병합합니다.

기술 스택

특수하거나 폐쇄적인 기술을 쓰지 않았습니다. 고객사 개발팀이 이어받아 유지보수할 수 있는 **표준 스택**으로 구성되어 있습니다.

레이어	구성
프론트엔드	Vue 3.5 · Vite · Pinia · Vue Flow(그래프 캔버스) · bpmn-js · Mermaid · Tailwind CSS · CodeMirror 6 · xterm.js
백엔드	Python 3.11+ · FastAPI · Uvicorn · Pydantic v2 · SSE(서버 전송 이벤트)
그래프	Neo4j 5 · Cypher · APOC. 스키마(제약·인덱스)는 파일로 형상관리
AI 오케스트레이션	LangChain · LangGraph · Deep Agents — 공급자 무관 인터페이스
LLM 공급자	Anthropic · OpenAI · Google · OpenRouter · OpenAI 호환 사내 게이트웨이
디자인 렌더	open-pencil 엔진 · CanvasKit(Skia) · Yoga 레이아웃 · Figma <code>.fig</code> 입출력
코딩 에이전트	Claude Code(PTY 세션) · MCP 서버 · git worktree 샌드박스
데스크톱	Electron 31 · electron-builder(dmg · nsis · Applmage) · 자동 업데이트
컨테이너	Podman(rootless · 데몬리스) 또는 Docker · Compose 호환
검증	pytest · Playwright · ruff — 정적 분석 · 단위 · E2E

세 가지 설계 원칙

공급자 종립

LLM 공급자와 모델은 환경 설정으로 바꿉니다. 특정 공급자에 종속되지 않도록 연동을 구성했습니다.

기능별 모듈

백엔드 `api/features/<이름>` 과 프론트엔드 `features/<이름>` 이 1:1로 대응하며, 기능 간 직접 참조를 금지합니다.

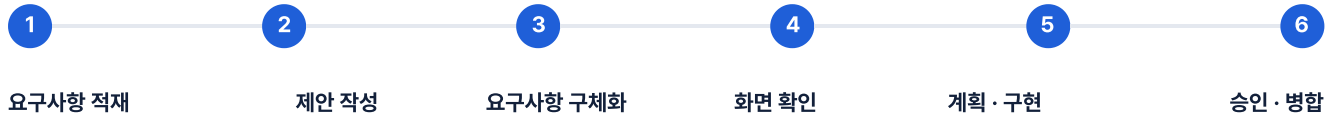
스킬 우선

방법론과 절차는 스킬 파일로 관리합니다. 절차를 바꾸는 데 제품 재배포가 필요 없습니다.

렌더 엔진은 오픈소스입니다. 화면 생성은 open-pencil(오픈소스 디자인 에디터) 엔진을 통해 이뤄지며, 상위 저장소와 정기적으로 동기화합니다. 렌더링 결과는 Figma로 왕복 가능한 표준 `.fig` 구조를 따릅니다.

사용 절차 — 여섯 단계

아래 화면은 모두 **실행 중인 제품에서 캡처한** 것입니다. 설명을 위한 목업이 아닙니다.



1단계 · 요구사항을 그래프로 올린다

기획 문서를 업로드하거나 Confluence 페이지를 연결하면, 21단계 처리 과정을 거쳐 유저스토리 · 컨텍스트 · Aggregate · 명령 · 이벤트 · 화면을 추출해 그래프에 적재합니다. 진행 상황이 단계별로 표시되며, 중간에 멈춰 검토한 뒤 이어갈 수 있습니다. 기존 시스템이 이미 있다면 이 단계를 건너뛰고 완성된 설계를 가져올 수도 있습니다.

2단계 · 바꾸고 싶은 것을 한 문단으로 적는다

Proposals 탭에서 새 제안을 만들고, 바꾸고 싶은 것을 자연어로 적습니다. 이때 **분해 모드**(간편 · DDD 상세 · ODA 표준)를 고릅니다.

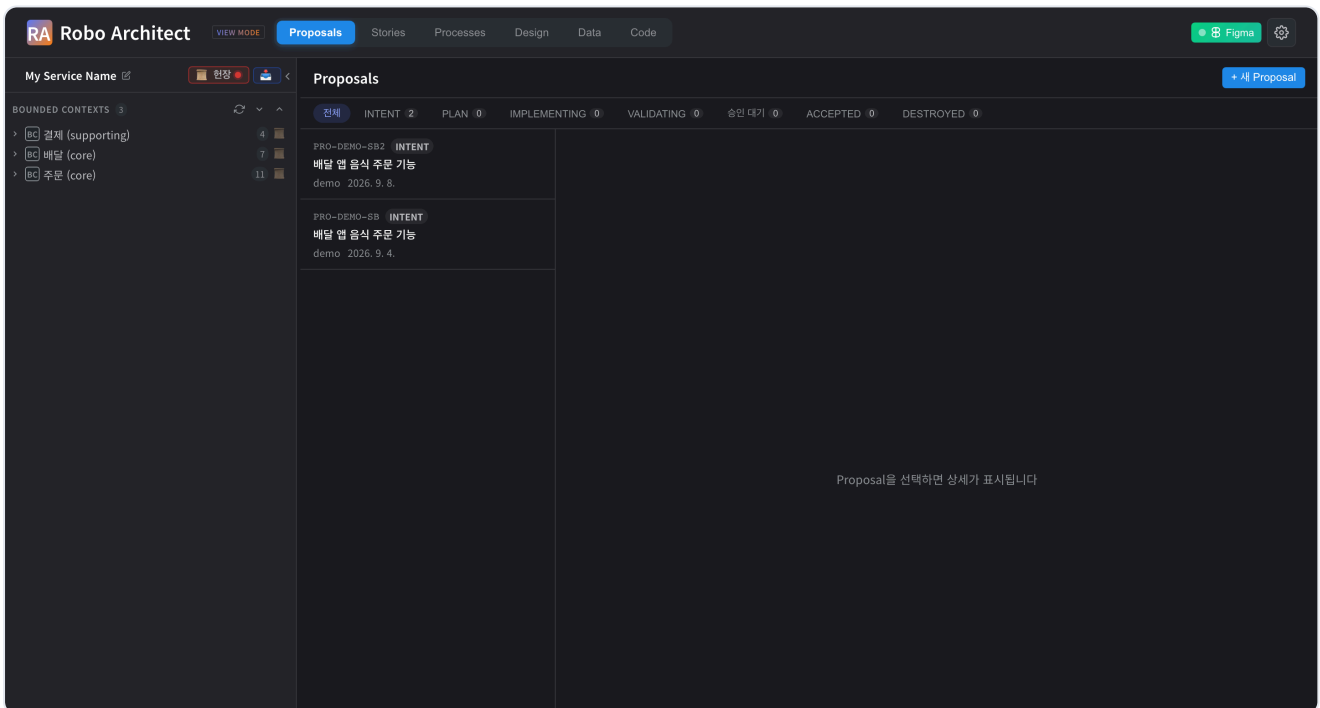


그림 9. 제안 목록 — 상태별 필터(Intent · Plan · Implementing · Validating · 승인 대기 · Accepted · Destroyed)로 진행 중인 변경을 한눈에 봅니다. *데모 데이터 기준입니다.*

3단계 · 요구사항 구체화 결과 검토

AI가 입력한 요구사항을 Epic · Feature · UserStory · 사용자 흐름으로 구체화합니다. 내용이 모호하면 선택형 질문으로 확인합니다. 검토 결과 수정이 필요하면 의견을 적어 다시 생성하거나 직접 수정할 수 있습니다.

4-6단계 · 화면 확인 · 구현 · 병합

4단계 · 화면으로 확인한다

요구사항 구체화가 끝나면 **스토리보드가 자동으로 생성**됩니다. 사용자 흐름의 화면들이 순서대로 그려지므로, 이해관계자는 텍스트가 아니라 화면을 보고 초안을 판단합니다.

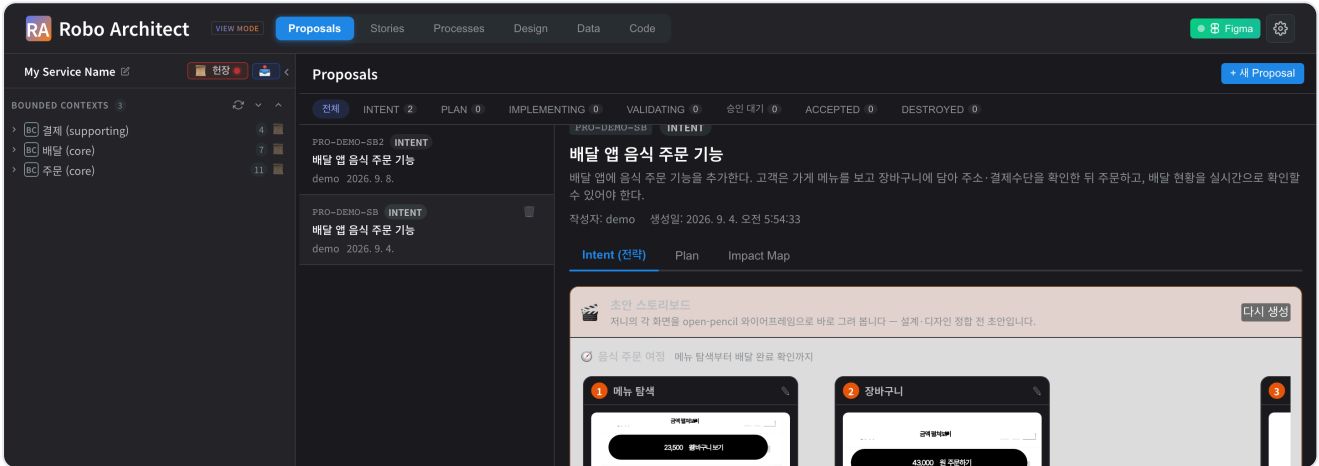


그림 10. 제안 상세 — 상단 탭(Intent · Plan · Impact Map)과 초안 스토리보드. 화면 카드를 클릭하면 편집기가 열립니다. 데모 데이터 기준입니다.

5단계 · 계획을 확정하고 구현한다

Plan

확정된 요구사항과 프로젝트 설계 원칙을 근거로 전술 설계(Aggregate · 명령 · 이벤트)와 배포 · 연동 · 저장소 구성 계획이 나옵니다.

Implement

대상 프로젝트에 격리된 작업 공간이 만들어지고, Code 탭에서 Claude Code가 구현을 수행합니다. 진행 로그를 보며 중지하거나 방향을 바꿀 수 있습니다.

6단계 · 검증하고 병합한다

인수조건(GWT)을 기준으로 결과를 검증한 뒤 승인하면, **코드 병합과 그래프 반영이 함께** 일어납니다. 갱신된 그래프는 최신 설계를 담고, 다음 변경의 기준이 됩니다.

검증

인수조건 기준 판정과 구조 검사 결과를 함께 봅니다.

병합

코드와 그래프를 한 흐름으로, 실패 시 양쪽 롤백.

이력

누가 언제 무엇을 승인했는지 상태 이력으로 남습니다.

중단해도 진행 상태가 보존됩니다. 문서 수집·분석, 분해, 구현 모두 중간 상태가 저장되어 화면을 나갔다 와도 이어서 진행됩니다. 페이지를 새로고침한 뒤에도 기존 구현 세션에 다시 연결할 수 있습니다.

보안 · 거버넌스

"AI가 설계를 바꾸고 코드를 쓴다"는 구조는 그 자체로 통제가 필요합니다. Robo Architect는 **설치형**을 전제로, 데이터가 나가지 않는 것과 변경이 통제되는 것 두 가지를 설계 기준으로 삼았습니다.

데이터 주권

- **설치형 제품입니다.** 클라우드 SaaS가 아니라 고객 인프라(또는 담당자 PC)에 설치됩니다. 설계 그래프와 소스코드는 그 경계를 벗어나지 않습니다.
- **LLM 엔드포인트를 지정합니다.** 상용 API 대신 사내 게이트웨이나 자체 호스팅 모델 (OpenAI 호환)을 지정할 수 있어, 프롬프트조차 외부로 나가지 않는 구성이 가능합니다.
- **모델은 교체 가능합니다.** 공급자·모델명이 환경 설정에 있으므로, 사내 정책이 바뀌면 설정만 바꿉니다.

변경 통제

항목	내용
승인 게이트	AI가 산출한 어떤 것도 사람의 명시적 확정 없이 그래프나 코드에 반영되지 않습니다
자기 승인 금지	변경을 제출한 사람이 그 변경을 승인할 수 없습니다
동시 편집 보호	버전 불일치 시 덮어쓰지 않고 충돌로 알립니다
코드 격리	구현은 전용 브랜치의 격리된 작업 공간에서만 수행하며, 원본 브랜치는 병합 전까지 변경되지 않습니다
되돌리기	승인 취소 시 설계 그래프의 반영을 되돌리며, 코드 되돌림 여부는 선택
감사 로그	요청별 상관관계 ID와 구조화 로그. 상태 전이 이력이 노드에 남습니다

조직의 설계 기준을 작업 절차에 반영

구체화 · 계획 · 구현 · 검증 절차가 스킴 파일에 정의돼 있으므로, 조직의 설계 규칙(명명 규칙, 필수 산출물, 금지 패턴)을 스킴에 반영하면 **모든 제안이 같은 기준을 거치도록** 할 수 있습니다. 프로젝트 설계 원칙에는 기술 스택 · 아키텍처 스타일 · 저장소 구성 전략이 저장되어 모든 계획에 적용됩니다.

도입 전 보안 검토를 권장합니다. 위 통제는 제품이 기본 제공하는 것이며, 고객 보안 정책(망 분리 수준, 계정 연동 방식, 로그 수집 대상)에 맞춘 구성은 도입 단계에서 함께 확정합니다.

배포 · 납품 모델

별도의 서버 구축 없이 **인스톨러 하나**로 시작할 수 있습니다. Python · Node · 데이터베이스를 따로 설치할 필요가 없습니다.

구분	절차	소요
데스크톱	인스톨러 실행 → 컨테이너 런타임 · 그래프 DB · 백엔드까지 자동 기동	수십 분
서버 설치	Compose로 백엔드 · 그래프 DB · 렌더 서비스 기동 후 사내 도메인 연결	수십 분
폐쇄망	컨테이너 이미지를 파일로 반입 → 검증 → 로드 → 기동	반입 절차 별도
업그레이드	새 버전으로 교체 후 재기동. 그래프 스키마는 기동 시 자동 적용	분 단위

데스크톱 패키징

단일 인스톨러

Windows(nsis) · macOS(dmg) · Linux(ApplImage).
백엔드를 자식 프로세스로 띄우고 앱 종료 시 함께 정리합니다. 자동 업데이트를 지원합니다.

rootless 컨테이너

Podman을 사용해 데몬 없이, 관리자 권한 없이 구동합니다. 상용 라이선스가 필요한 데스크톱 컨테이너 제품에 의존하지 않습니다.

선택 설치 팩

레거시·데이터 분석 팩은 설치 시 선택하거나 나중에 추가·제거할 수 있습니다.

외부 DB 연결

그래프 데이터베이스를 별도로 설치하는 대신, 사내에서 운영 중인 데이터베이스에 연결할 수도 있습니다.

시스템 요구사항 (권장)

담당자 PC 기준 메모리 16GB 이상, 컨테이너 런타임, 그리고 LLM 접근 경로(사내 게이트웨이 또는 외부 API). 서버 설치 시에는 동시 사용자 수를 기준으로 별도 산정해 드립니다.

산출물은 고객 자산으로 남습니다. 설계 그래프는 표준 Cypher로 내보낼 수 있고 생성된 코드는 고객 저장소에 그대로 남으므로, 제품 사용을 종료한 뒤에도 설계 데이터와 코드를 보유하고 활용할 수 있습니다.

제품이 스스로를 검증하는 방식

"AI가 만든 설계를 믿을 수 있는가"에 대한 답을 제품 개발 방식으로 제시합니다. Robo Architect 개발팀은 **제품 개발에도 동일한 방법론**을 적용합니다.

스펙 선행 개발

모든 기능은 명세(spec) → 계획(plan) → 태스크(tasks) → 구현 순서로 진행되며, 그 산출물이 저장소에 함께 남습니다. 현재 **47개의 기능 명세**가 축적돼 있고, 각각에 무엇을 왜 그렇게 결정했는지가 기록돼 있습니다.

제품 개발 원칙 — 타협하지 않는 규칙

원칙	내용
I. 그래프가 기준	모든 설계 정보는 그래프에 저장하고 별도의 기준 데이터를 두지 않는다 (타협 불가)
II. 도메인 언어	CRUD 어휘 금지, 이벤트스토크 어휘 사용
III. 스트리밍 우선	수 초 이상 걸리는 작업은 진행률 스트리밍 필수
IV. 사람이 승인	계획과 적용을 분리. 자동 적용 금지
V. 기능별 모듈	백엔드와 프론트엔드의 기능 모듈을 1:1로 대응, 기능 간 직접 참조 금지
VI. 공급자 중립	LLM 공급자-모델은 설정으로. 하드코딩 금지
VII. 관측 가능	상관관계 ID + 구조화 로그를 기본으로
VIII. 화면 생성 경로	화면은 반드시 렌더 엔진을 거친다 (타협 불가)
IX. 플러그인 규율	Figma 플러그인과 백엔드 간 개발 루프 규칙
X. 스킴 우선	절차와 방법론은 스킴 파일로 관리하고 제품 코드에 포함하지 않는다

자동 검증

49

단위 테스트 모듈
기능별 pytest

70

E2E 시나리오
Playwright — 실제 브라우저

정적 분석

전 코드
ruff · 타입 검사

본 소개서에는 실제 제품 실행 화면을 수록했습니다. 모든 화면은 상시 가동 중인 환경에서 캡처했습니다.

기반 제품의 도입 실적과 확장

Robo Architect는 이벤트스토밍 기반 설계 · 코드 생성 도구 **MSAEZ(MSA Easy)**의 경험을 바탕으로 개발한 차세대 제품입니다. 아래는 기반 제품 MSAEZ가 산업 현장과 공공 플랫폼, 교육 과정에서 쌓아 온 도입 실적입니다.

대기업 실무

포스코DX

애플리케이션 · 마이크로서비스 설계와
소스코드 생성 도구로 사용 중

공공 플랫폼

DPG 통합테스트베드

첨단 디지털 개발지원 도구(SaaS)로 등록·
운영

9,655명

MSA School 누적 수강생

325회 · 17,864시간 교육에 표준 도구로
채택

① 산업 현장 — 포스코DX

포스코DX에서 **애플리케이션 및 마이크로서비스의 설계와 소스코드 생성 도구**로 사용되고 있습니다. 대규모 제조 · 산업 IT 조직의 실제 개발 과정에서, 이벤트스토밍으로 도출한 설계 모델을 기반으로 마이크로서비스 코드를 생성하는 데 활용됐습니다.

② 공공 — DPG 통합테스트베드

DPG(디지털플랫폼정부) 통합테스트베드는 한국지능정보사회진흥원(NIA)이 운영하는, **민간·공공의 데이터와 클라우드 자원을 활용해 혁신 서비스를 개발·시험·검증**할 수 있도록 지원하는 플랫폼입니다. 중소기업·스타트업·시민개발자·공공기관이 이용 대상이며, 인프라를 직접 구축하는 대신 민간이 개발한 **서비스형 소프트웨어(SaaS)**를 개발지원 도구로 제공합니다.

MSA Easy(MSAEZ)는 이 테스트베드에 등록되어 **마이크로서비스의 분석 · 설계 · 구현 · 배포 전 과정을 자동화**하는 SaaS로 제공되고 있습니다.

③ 교육 — MSA School

클라우드 네이티브 애플리케이션 개발 전문 교육기관 **MSA School**의 표준 실습 도구로 채택되어, 수강생이 이벤트스토밍으로 도메인을 설계하고 그 모델에서 코드를 생성해 배포하는 전 과정을 실습했습니다.

누적 실적은 **교육 325회 · 수강생 9,655명 · 17,864시간**이며, 커리큘럼은 기초 · 중급 · 고급 3단계로 Biz(이벤트스토밍 · DDD) · Dev(Spring Boot · Kafka) · Ops(Kubernetes · GitOps) 전 영역을 다룹니다.

Robo Architect가 이어받은 것과 새로 더한 것. 이벤트스토밍 설계와 코드 생성이라는 검증된 축은 그대로 있고, 여기에 **변경 영향 전파 · 제안 생애주기 · 초안 스토리보드 · 코딩 에이전트 통합 · 폐쇄망 설치형**을 더했습니다. Robo Architect는 **초기 설계부터 이후 변경 관리까지** 지원합니다.

* 수치는 MSA School 공개 집계 기준이며, 상세 구축 범위와 도입 효과는 담당 영업 채널을 통해 개별 안내드립니다.

도입 절차 및 일정

PoC부터 본 도입까지 **4단계**로 진행합니다. 아래는 표준 일정이며, 고객사 환경과 검토 절차에 따라 조정됩니다.



PoC 주요 검증 항목

- 실제 요구사항 문서의 주요 개념과 관계가 **도메인 그래프에 적절히 반영**되는지
- 실제 변경 사례에 대해 **관련 설계 요소와 소스 파일**을 식별하는지
- 생성된 화면 초안으로 **이해관계자가 기능과 사용 흐름**을 이해할 수 있는지
- 기존 저장소 구성과 브랜치 운영 방식에 맞춰 **구현 · 병합**을 진행할 수 있는지
- 폐쇄망 설치와 사내 LLM 연동이 **고객사의 보안 정책**을 준수하는지

도입 단계에서 함께 정할 것

LLM 실행 경로

외부 상용 API · 사내 게이트웨이 · 자체 호스팅 중 무엇을 쓸지.

대상 저장소 · 브랜치 전략

격리된 작업 공간을 어느 저장소에 만들고 어느 브랜치로 병합할지.

조직 공통 설계 기준

기술 스택 · 아키텍처 스타일 · 저장소 구성 전략을 한 번 수립.

스킬 커스터마이징

조직 고유 설계 절차 · 산출물 양식을 스킬로 반영.

다음 단계 — 30분 데모 · PoC 환경 협의

실행 중인 제품에서 "요구사항 한 문단 → 설계 그래프 → 화면"까지 직접 보시는 것이 가장 빠릅니다. 고객사 도메인 문서 일부를 미리 주시면 **귀사 데이터**로 시연해 드립니다.



Robo Architect

AI ARCHITECTURE WORKBENCH

도입 문의

요구사항부터 구현까지, 변경을 함께 관리하세요.

도입 상담 · 데모 요청 · 견적 문의 — 무엇이든 남겨주세요. 담당 전문가가 영업일 기준
1일 이내에 답변드리며, 원하시면 실제 제품을 활용한 시연 일정도 안내해 드립니다.

회사

(주)유엔진솔루션즈

제품

Robo Architect

문서

Rev. 2026.09

한 흐름으로 이어지는 네 단계

요구사항

문서 · 대화에서 그래프로

설계

DDD · 이벤트스토밍

화면

초안부터 Figma까지

코드

격리 환경 구현 · 병합

(주)유엔진솔루션즈 · uEngine Solutions, Inc.

© 2026 uEngine Solutions. 본 문서 Rev. 2026.09 · 기재된 수치 · 화면은 작성 시점 기준이며 사전 통지 없이 변경될 수 있습니다.

uEngine